

Half Adder

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

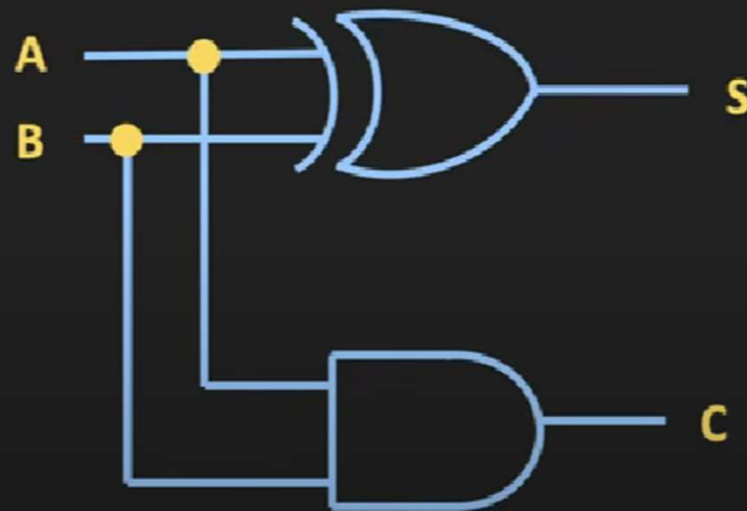
$$C = AB$$

Half Adder

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

Logic Circuit



</> verilog

```
module half_adder(  
    input A, B,  
    output Sum, Carry  
);  
    assign Sum = A ^ B;  
    assign Carry = A & B;  
endmodule
```

Full Adder

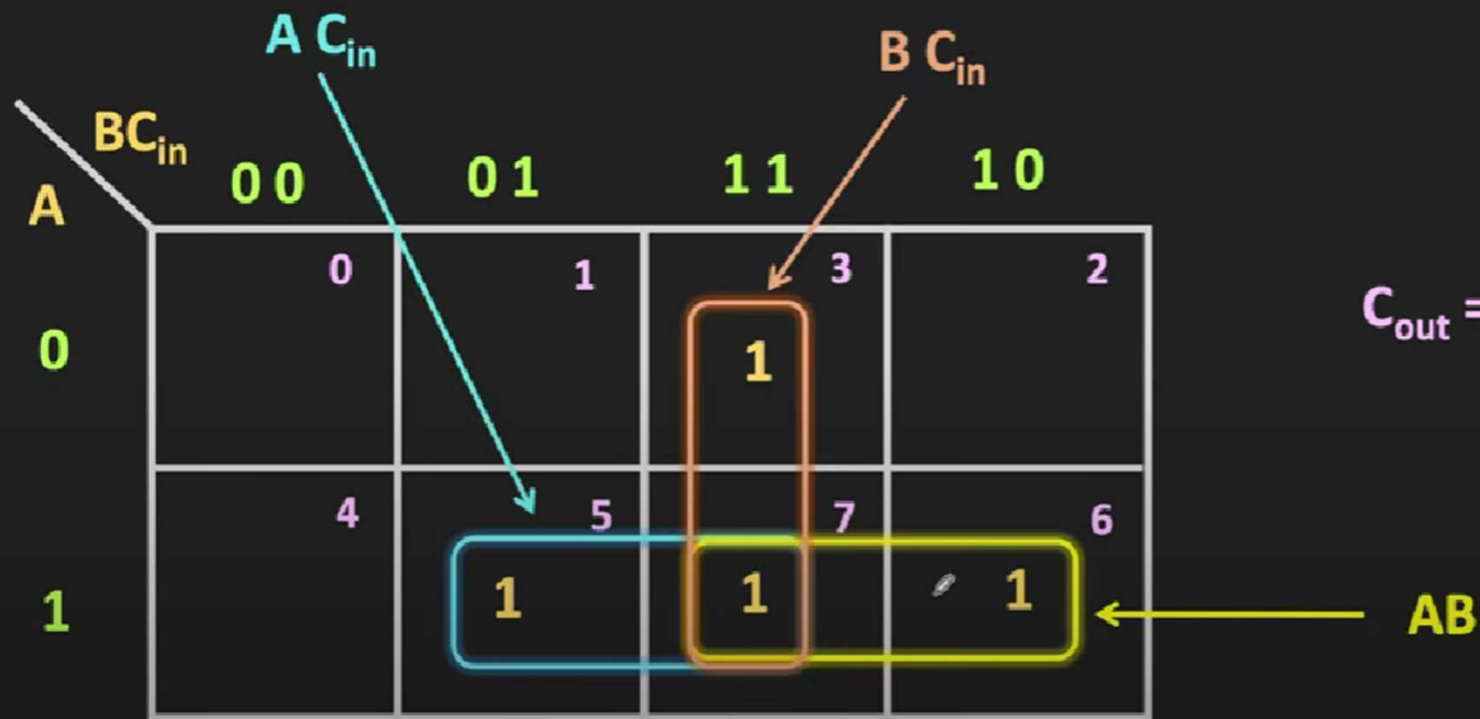
A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned}C_{out} &= \overline{A} B C_{in} + A \overline{B} C_{in} + A B \overline{C}_{in} + A B C_{in} \\&= A B (C_{in} + \overline{C}_{in}) + C_{in} (\overline{A} B + A \overline{B}) \\&= A B + C_{in} (A \oplus B)\end{aligned}$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

Full Adder

$$C_{out} = \overline{A} B C_{in} + A \overline{B} C_{in} + A B \overline{C}_{in} + A B C_{in}$$



$$C_{out} = AB + BC_{in} + AC_{in}$$

Full Adder

A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = \overline{A} \overline{B} C_{in} + \overline{A} B \overline{C}_{in} + A \overline{B} \overline{C}_{in} + A B C_{in}$$

$$= \overline{A} (\overline{B} C_{in} + B \overline{C}_{in}) + A (\overline{B} \overline{C}_{in} + B C_{in})$$

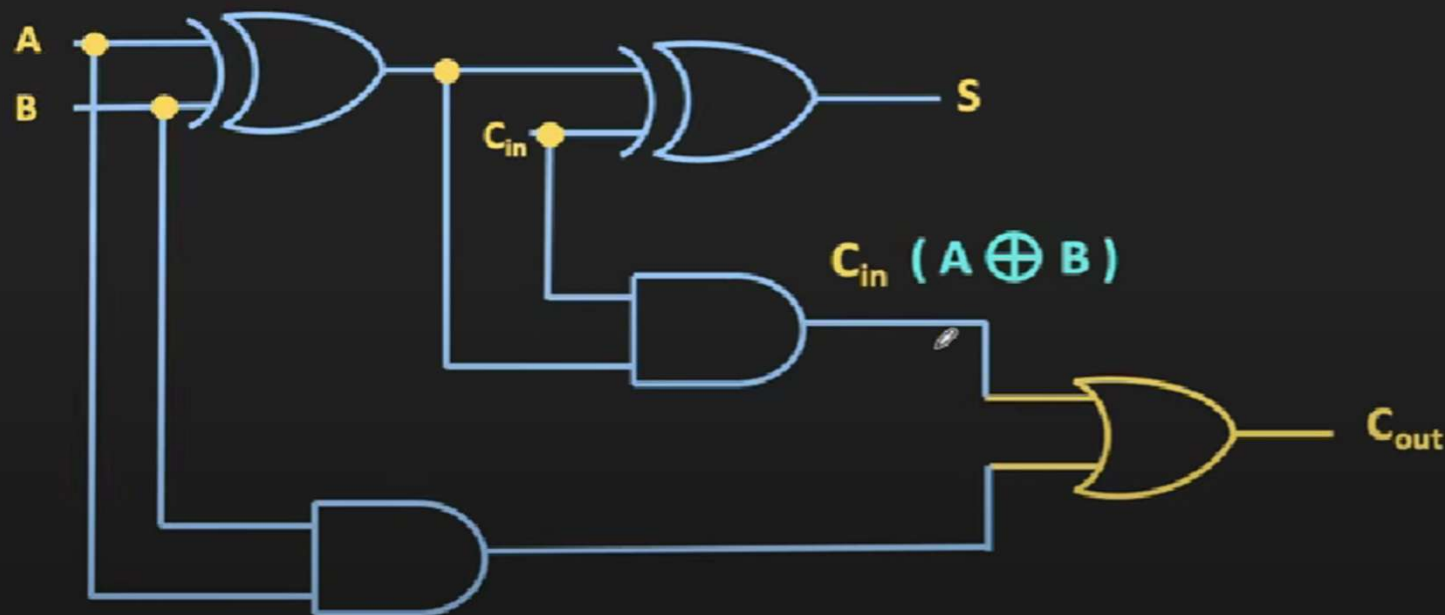
$$= \overline{A} (\underbrace{B \oplus C_{in}}_X) + A (\overline{B \oplus C_{in}})$$

$$S = A \oplus B \oplus C_{in}$$

Full Adder

$$S = A \oplus B \oplus C_{in}$$

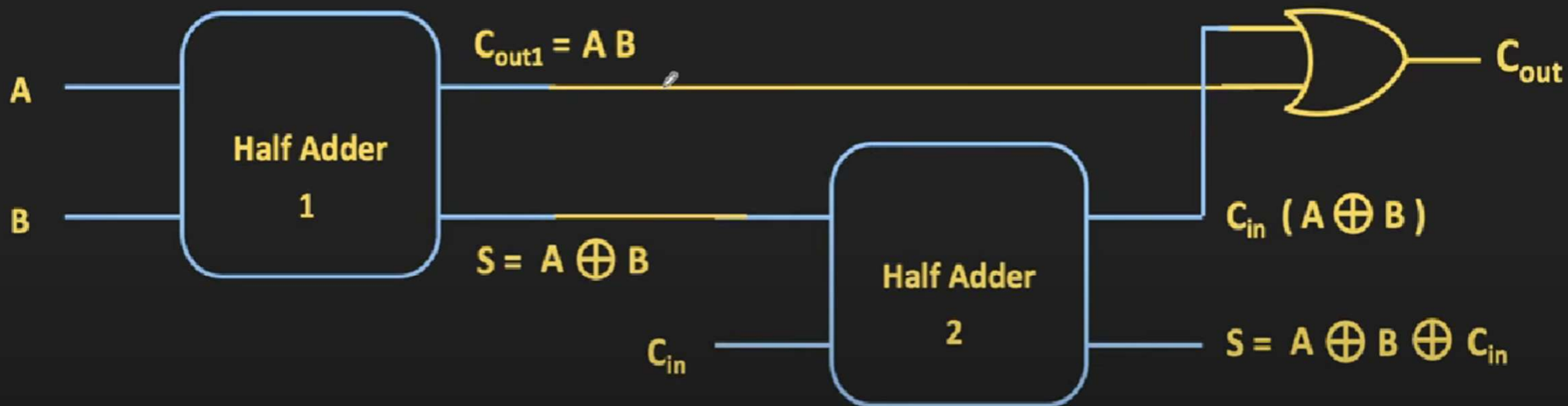
$$C_{out} = AB + BC_{in} + AC_{in} = AB + C_{in}(A \oplus B)$$



Full Adder using Half Adders

$$S = A \oplus B \oplus C_{in}$$

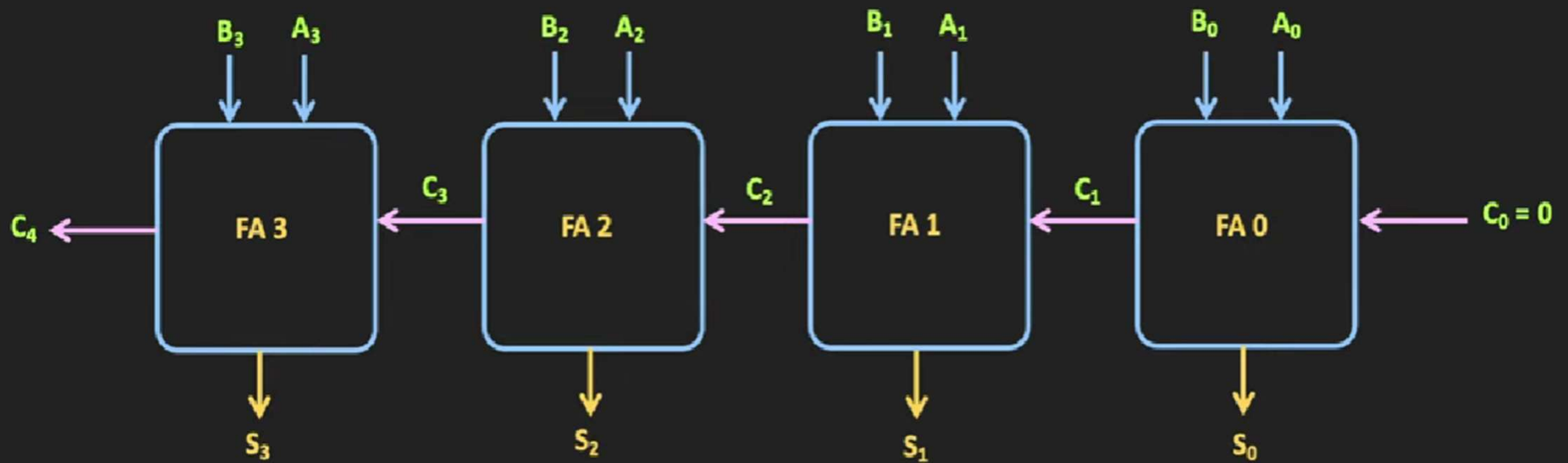
$$C_{out} = AB + C_{in} (A \oplus B)$$



</> verilog

```
module full_adder(  
    input A, B, Cin,  
    output Sum, Cout  
);  
    assign Sum = A ^ B ^ Cin;  
    assign Cout = (A & B) | (B & Cin) | (A & Cin);  
endmodule
```

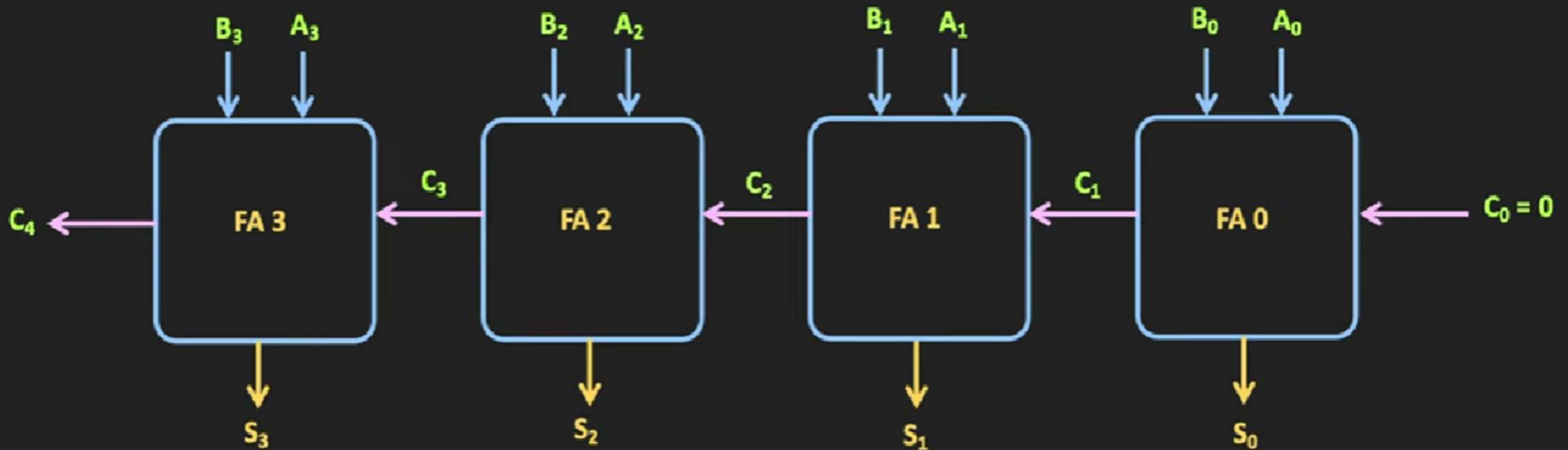
4-bit Adder



t_s – Propagation Delay for Sum output

t_c – Propagation Delay for Carry output

4-bit Adder



If $t_s \neq t_c$, then for n -bit adder,

Total Propagation Delay for Sum output = $(n - 1) t_c + t_s$

Total Propagation Delay for Carry output = $n t_c$

```
</> verilog
```

```
module ripple_carry_adder(
```

```
    input [3:0] A, B,
```

```
    input Cin,
```

```
    output [3:0] Sum,
```

```
    output Cout
```

```
);
```

```
    wire C1, C2, C3;
```

```
    full_adder FA0(A[0], B[0], Cin, Sum[0], C1);
```

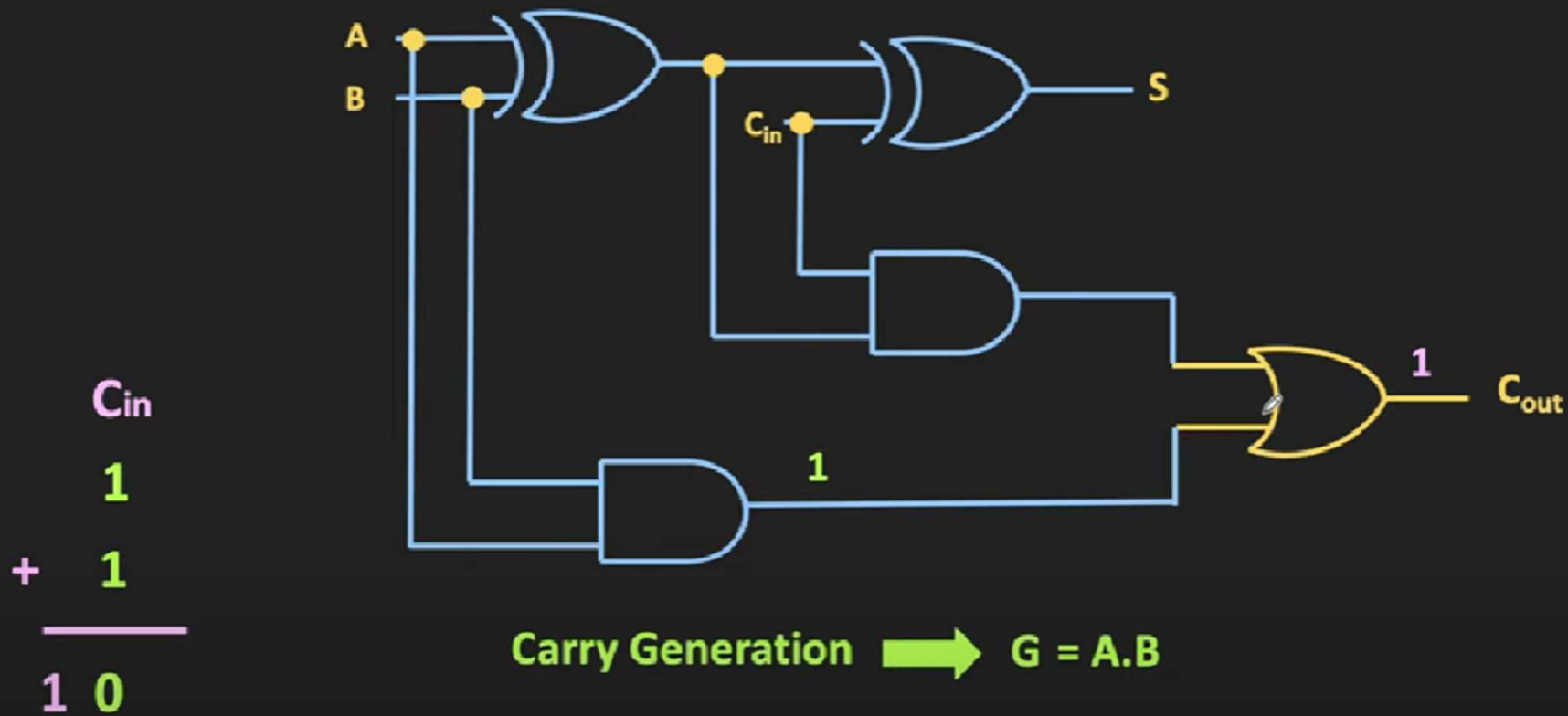
```
    full_adder FA1(A[1], B[1], C1, Sum[1], C2);
```

```
    full_adder FA2(A[2], B[2], C2, Sum[2], C3);
```

```
    full_adder FA3(A[3], B[3], C3, Sum[3], Cout);
```

```
endmodule
```

Carry Generation and Propagation



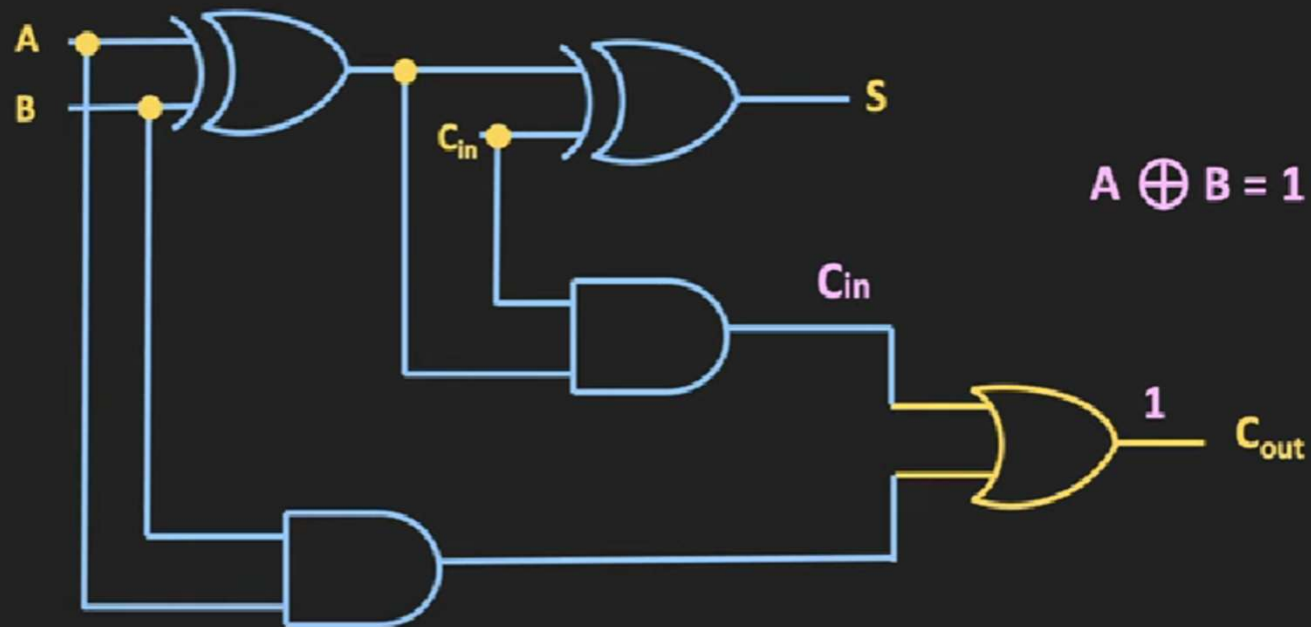
Carry Generation and Propagation

$C_{in} = 1$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 10 \end{array}$$

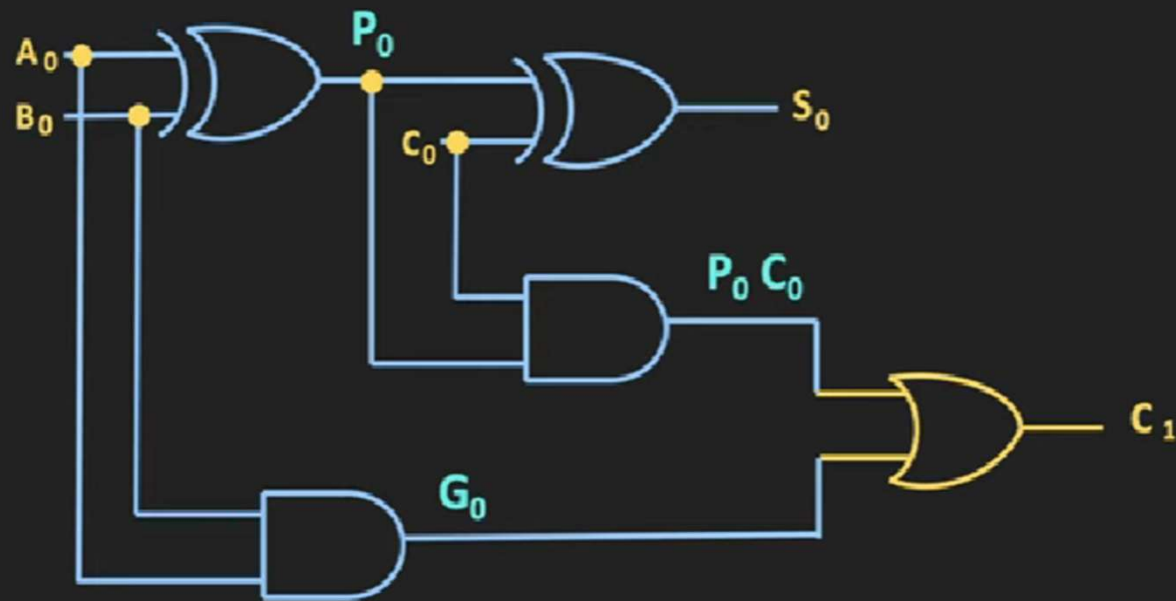
$C_{in} = 1$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 10 \end{array}$$

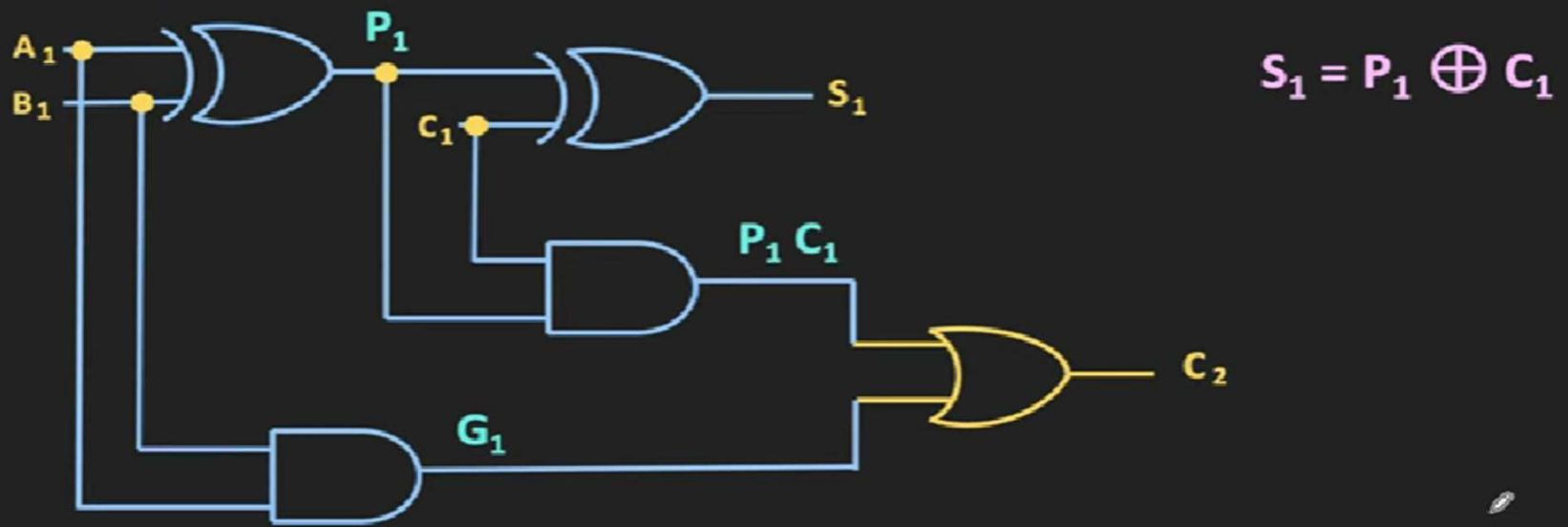


Carry Propagation $\rightarrow P = A \oplus B$

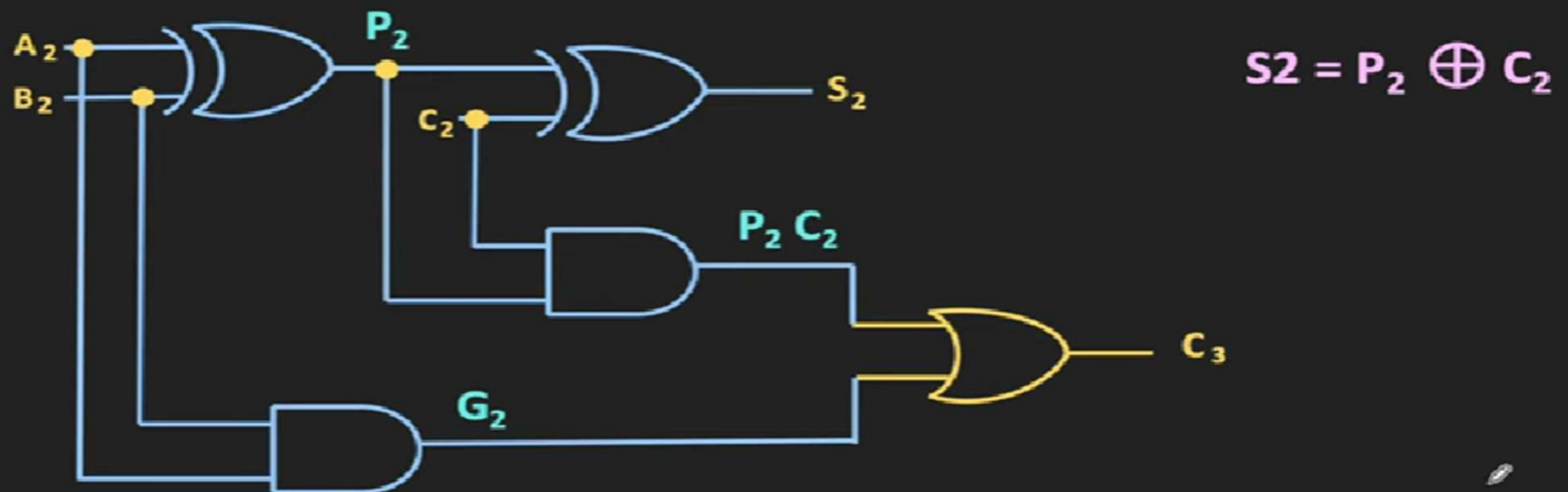
Carry Generation and Propagation



Carry Generation and Propagation



Carry Generation and Propagation

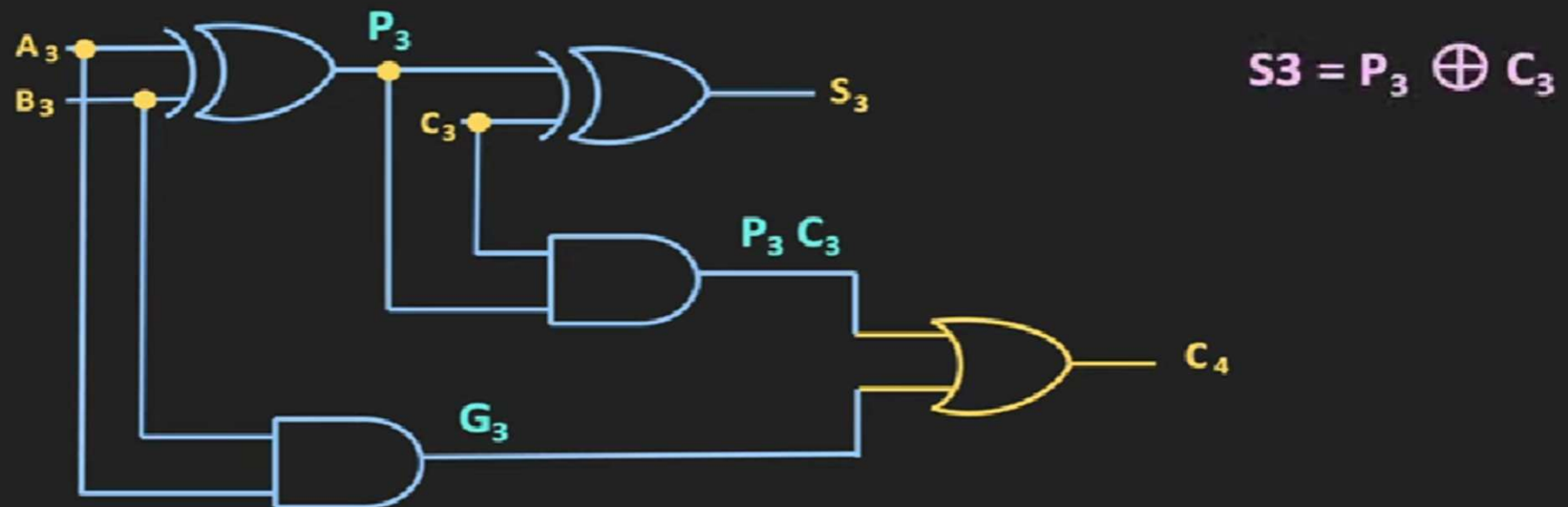


$$C_3 = G_2 + P_2 C_2$$

$$= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0)$$

$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

Carry Generation and Propagation



$$S_3 = P_3 \oplus C_3$$

$$C_4 = G_3 + P_3 C_3$$

$$= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0)$$

$$= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

4-bit Carry Look Ahead Adder

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

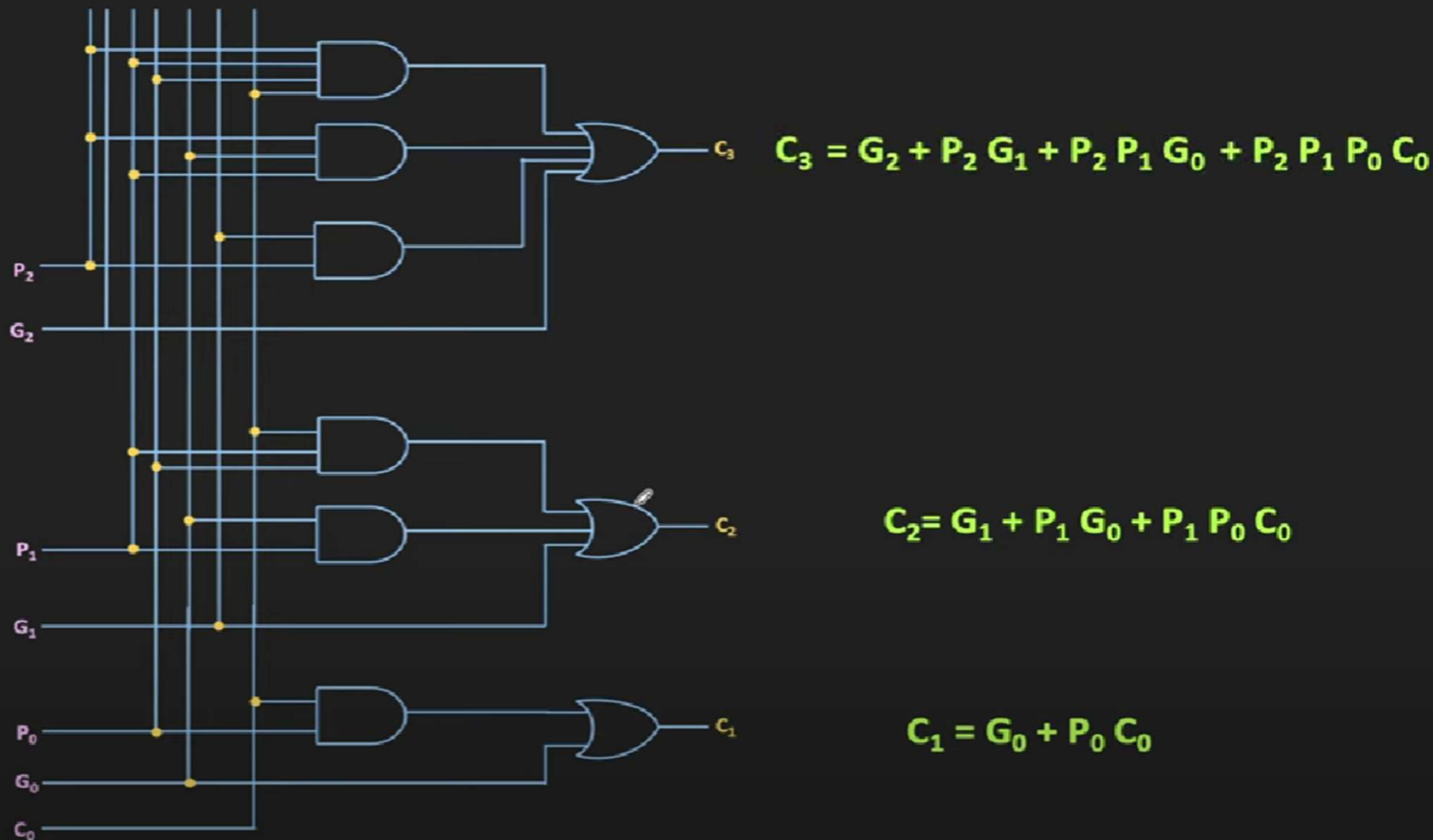
$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

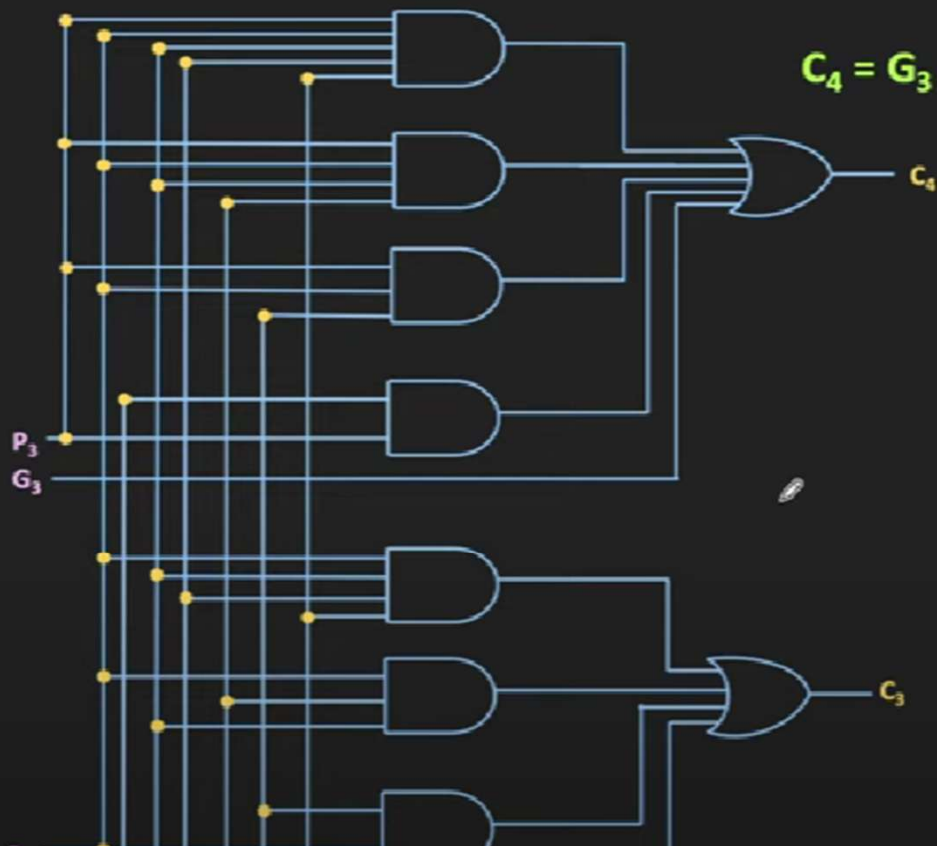
$$G_n = A_n B_n$$

$$P_n = A_n \oplus B_n$$

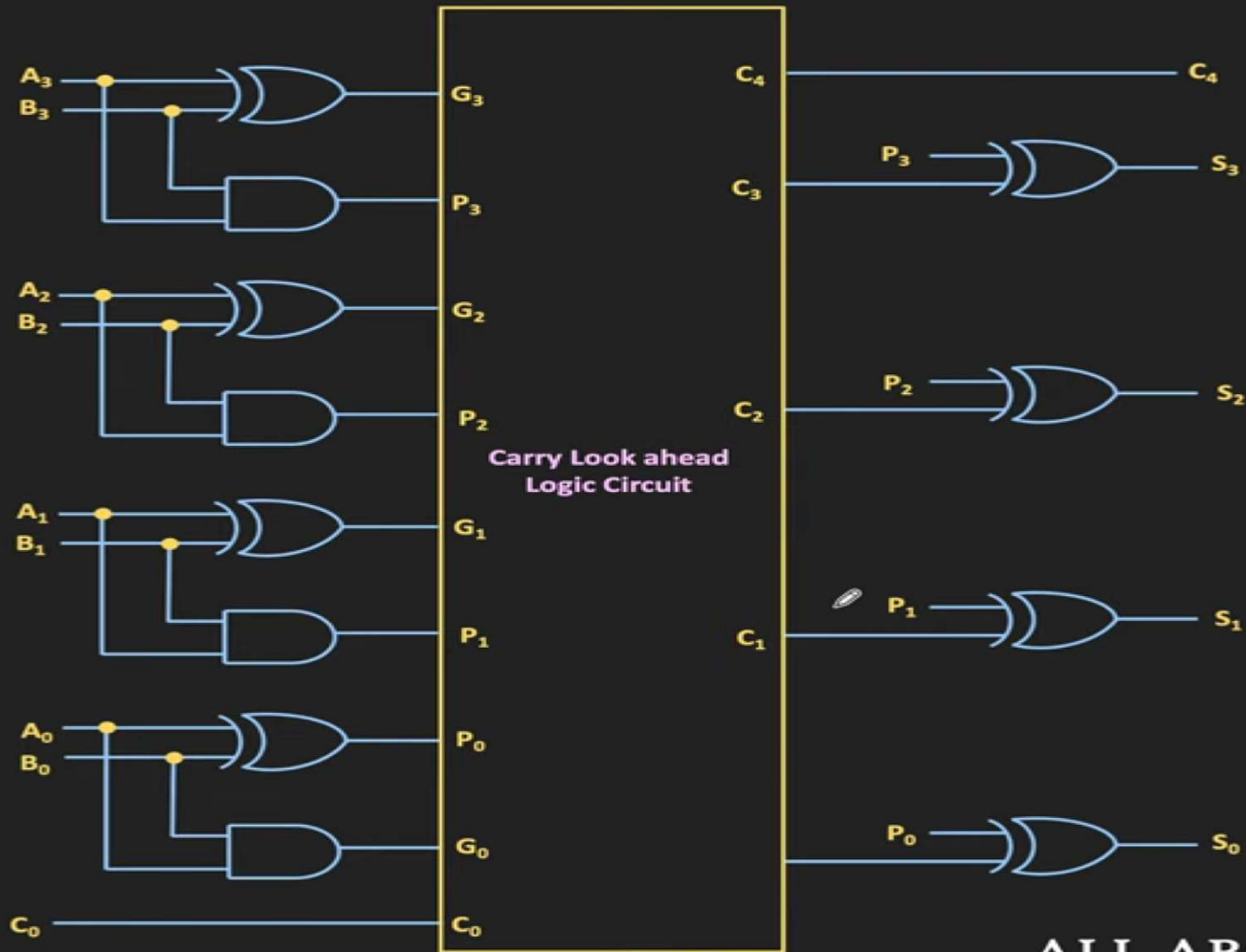
Look Ahead Carry Adder logic circuit



Look Ahead Carry Adder logic circuit



4-bit Look Ahead Carry Adder



$$S_n = P_n \oplus C_n$$

```

module carry_lookahead_adder(
    input [3:0] A, B,
    input Cin,
    output [3:0] Sum,
    output Cout
);
    wire [3:0] G, P;
    wire C1, C2, C3;

    // Generate and Propagate
    assign G = A & B;
    assign P = A ^ B;

    // Carry equations
    assign C1 = G[0] | (P[0] & Cin);
    assign C2 = G[1] | (P[1] & G[0]) | (P[1] & P[0] & Cin);
    assign C3 = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & Cin);
    assign Cout = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) |
        (P[3] & P[2] & P[1] & G[0]) |
        (P[3] & P[2] & P[1] & P[0] & Cin);

    // Sum
    assign Sum[0] = P[0] ^ Cin;
    assign Sum[1] = P[1] ^ C1;
    assign Sum[2] = P[2] ^ C2;
    assign Sum[3] = P[3] ^ C3;

endmodule

```